

MANUAL LABORATORIO1

Primer problema detectado, no me dejaba hacer Git : Clone en VS, he instalado una extensión de Git(Git Extension).

1º Paso

Instalar el Managment Studio 22, una vez instalado e iniciado el servidor en localhost, tiro el primer script de la tardea para crear la bbdd:

```
CREATE DATABASE EcommerceDB; GO
```

```
USE EcommerceDB; GO
```

2º Paso

Tirar el script de la creación de tablas. El use EcommerceDB sirve para trabajar sobre esa bbdd.####Script de creacion de tablas: USE EcommerceDB;

```
GO
```

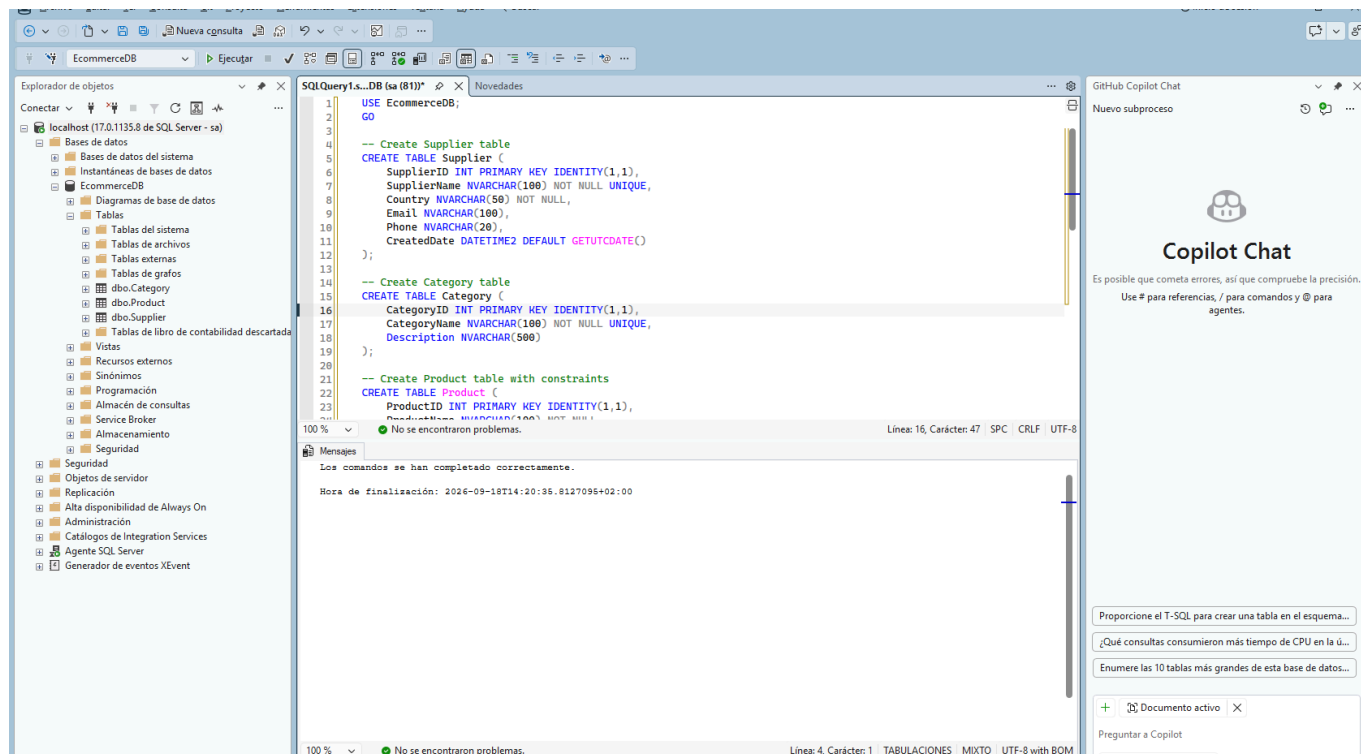
```
-- Create Supplier table CREATE TABLE Supplier ( SupplierID INT PRIMARY KEY IDENTITY(1,1), SupplierName NVARCHAR(100) NOT NULL UNIQUE, Country NVARCHAR(50) NOT NULL, Email NVARCHAR(100), Phone NVARCHAR(20), CreatedDate DATETIME2 DEFAULT GETUTCDATE() );
```

```
-- Create Category table CREATE TABLE Category ( CategoryID INT PRIMARY KEY IDENTITY(1,1), CategoryName NVARCHAR(100) NOT NULL UNIQUE, Description NVARCHAR(500) );
```

```
-- Create Product table with constraints CREATE TABLE Product ( ProductID INT PRIMARY KEY IDENTITY(1,1), ProductName NVARCHAR(100) NOT NULL, CategoryID INT NOT NULL, SupplierID INT NOT NULL, BasePrice DECIMAL(10,2) NOT NULL, StockQuantity INT NOT NULL DEFAULT 0, CreatedDate DATETIME2 DEFAULT GETUTCDATE(), CHECK (BasePrice > 0), CHECK (StockQuantity >= 0), FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID), FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID), );
```

```
-- Create indexes CREATE INDEX IX_Category ON Product(CategoryID); CREATE INDEX IX_Supplier ON Product(SupplierID);
```

GO



3º Paso

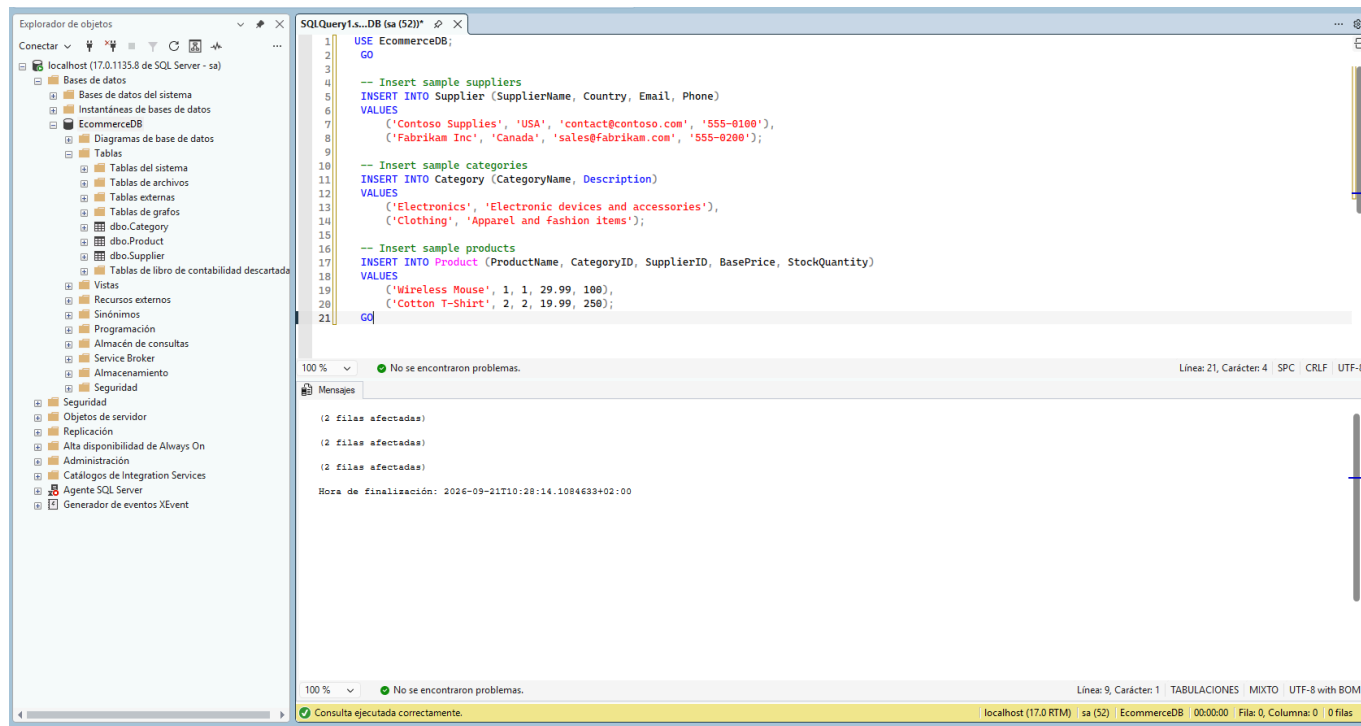
Insertamos datos en las tablas

USE EcommerceDB; GO

-- Insert sample suppliers INSERT INTO Supplier (SupplierName, Country, Email, Phone) VALUES ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'), ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');

-- Insert sample categories INSERT INTO Category (CategoryName, Description) VALUES ('Electronics', 'Electronic devices and accessories'), ('Clothing', 'Apparel and fashion items');

-- Insert sample products INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity) VALUES ('Wireless Mouse', 1, 1, 29.99, 100), ('Cotton T-Shirt', 2, 2, 19.99, 250); GO



4º Paso

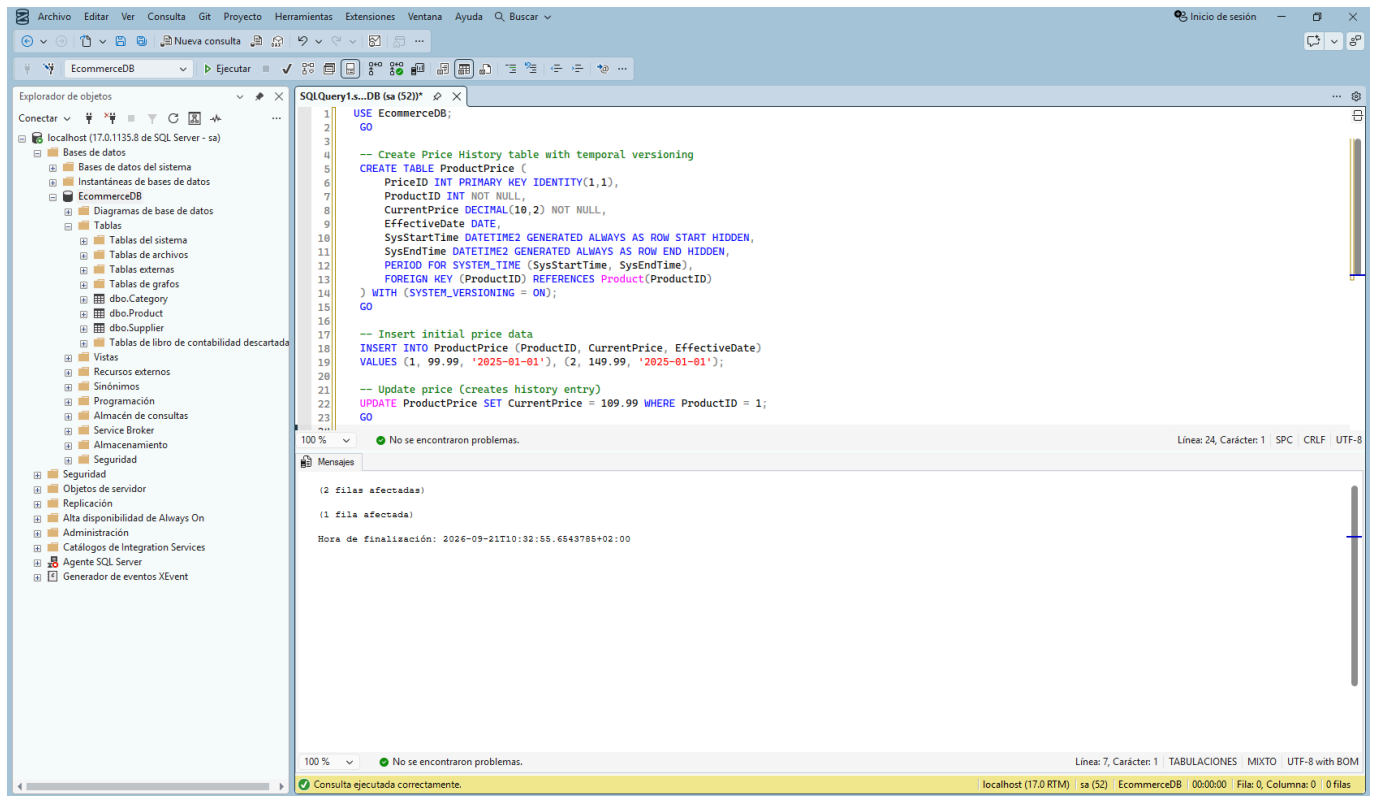
Creamos una tabla temporal para el historial de precios para mantener un registro completo de los cambios de precio

USE EcommerceDB; GO

-- Create Price History table with temporal versioning CREATE TABLE ProductPrice (PriceID INT PRIMARY KEY IDENTITY(1,1), ProductID INT NOT NULL, CurrentPrice DECIMAL(10,2) NOT NULL, EffectiveDate DATE, SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN, SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN, PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime), FOREIGN KEY (ProductID) REFERENCES Product(ProductID)) WITH (SYSTEM_VERSIONING = ON); GO

-- Insert initial price data INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate) VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');

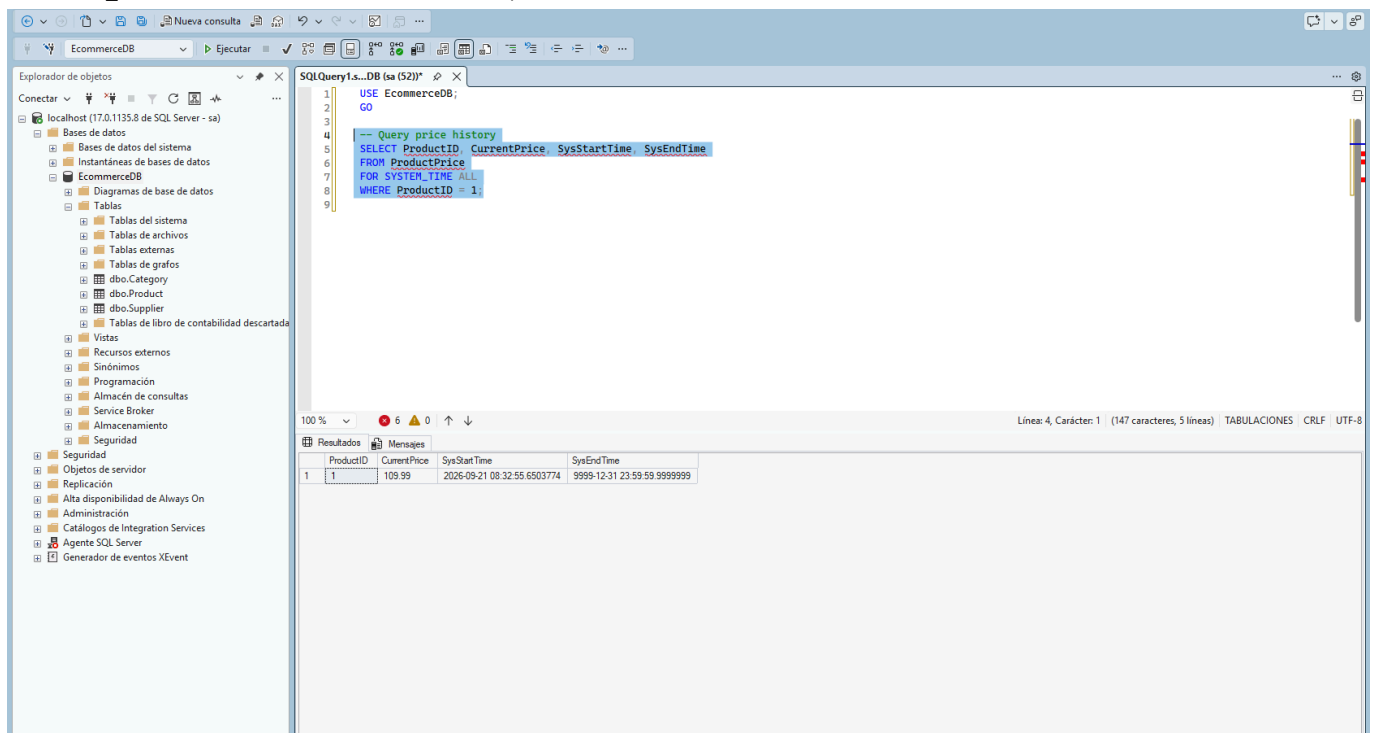
-- Update price (creates history entry) UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1; GO



Ahora hacemos un select de esta nueva tabla para comprobar que se ha creado correctamente

USE EcommerceDB; GO

-- Query price history SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime FROM ProductPrice FOR SYSTEM_TIME ALL WHERE ProductID = 1;



5º paso

Agregamos columnas JSON para metadatos. Las columnas JSON almacenan datos flexibles y variables que difieren según el tipo de producto.

USE EcommerceDB; GO

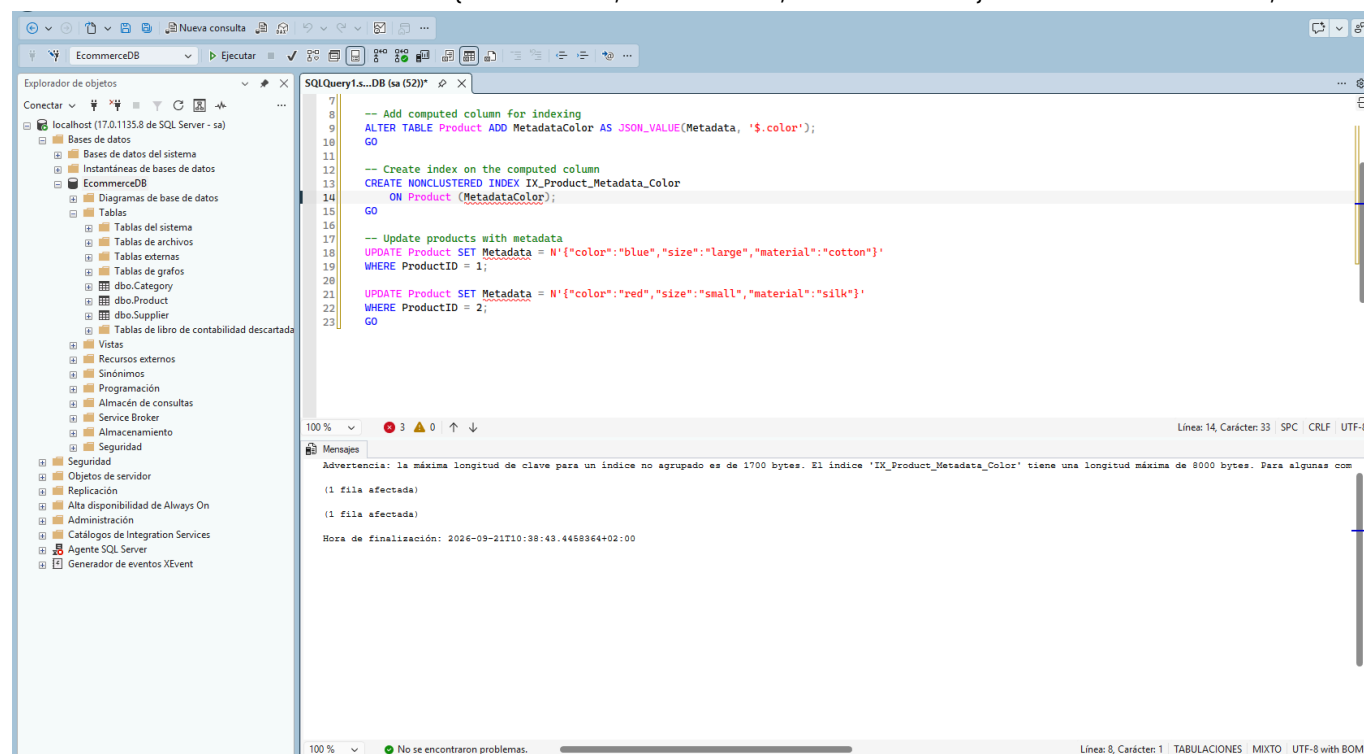
```
-- Add metadata column to Product (JSON type requires SQL Server 2025) ALTER TABLE Product ADD  
Metadata JSON; GO
```

```
-- Add computed column for indexing ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata,  
'$.color'); GO
```

```
-- Create index on the computed column CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color ON  
Product (MetadataColor); GO
```

```
-- Update products with metadata UPDATE Product SET Metadata =  
N'{"color":"blue","size":"large","material":"cotton"}' WHERE ProductID = 1;
```

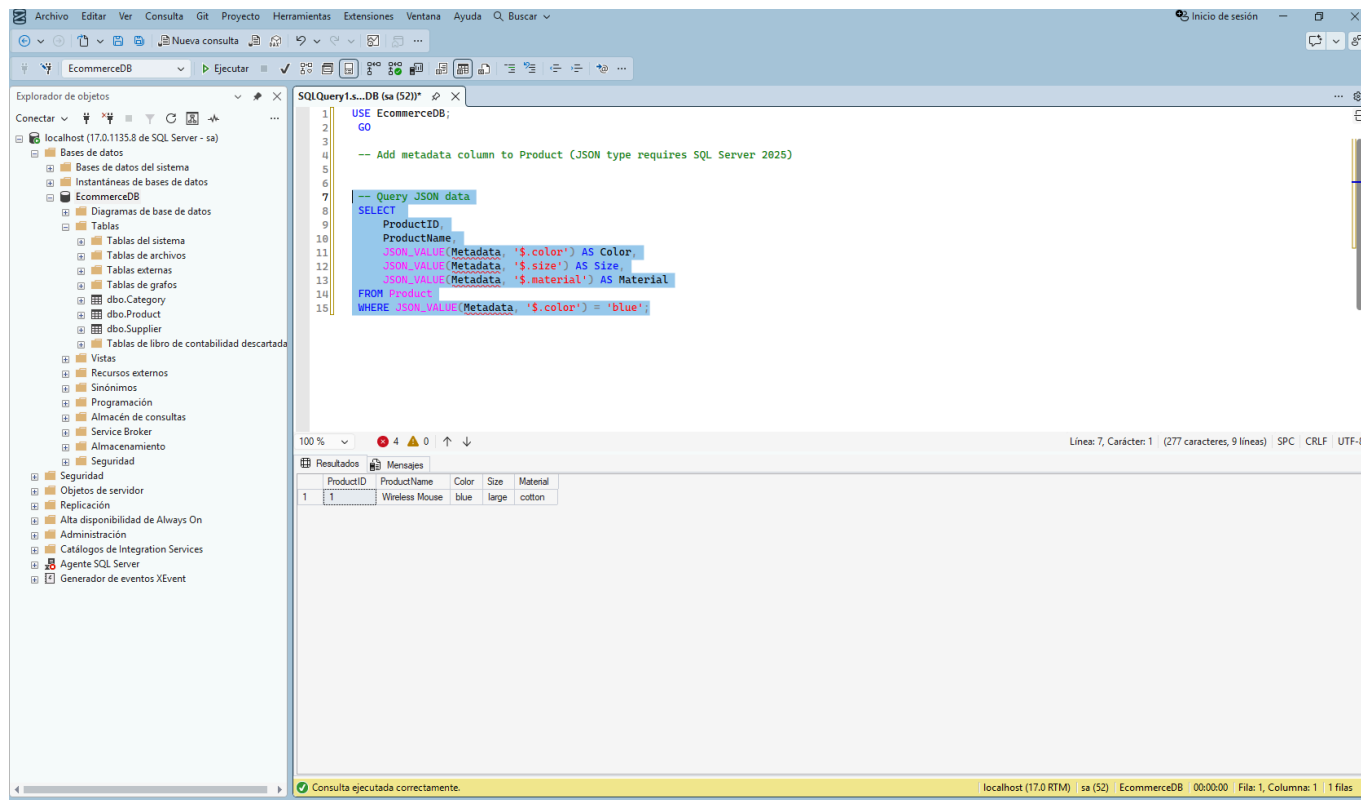
```
UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"silk"}' WHERE ProductID = 2; GO
```



Ahora consultamos las columnas JSON

USE EcommerceDB; GO

```
-- Query price history SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime FROM ProductPrice FOR  
SYSTEM_TIME ALL WHERE ProductID = 1;
```



6º paso

Creamos una nueva tabla de pedidos particionada. El particionamiento divide las tablas grandes en segmentos más pequeños para agilizar las consultas y facilitar el mantenimiento.

USE EcommerceDB; GO

-- Create partition function for order dates -- Use RANGE RIGHT for date columns to keep same-day values together
 CREATE PARTITION FUNCTION PF_OrderDate (DATE) AS RANGE RIGHT FOR VALUES ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');

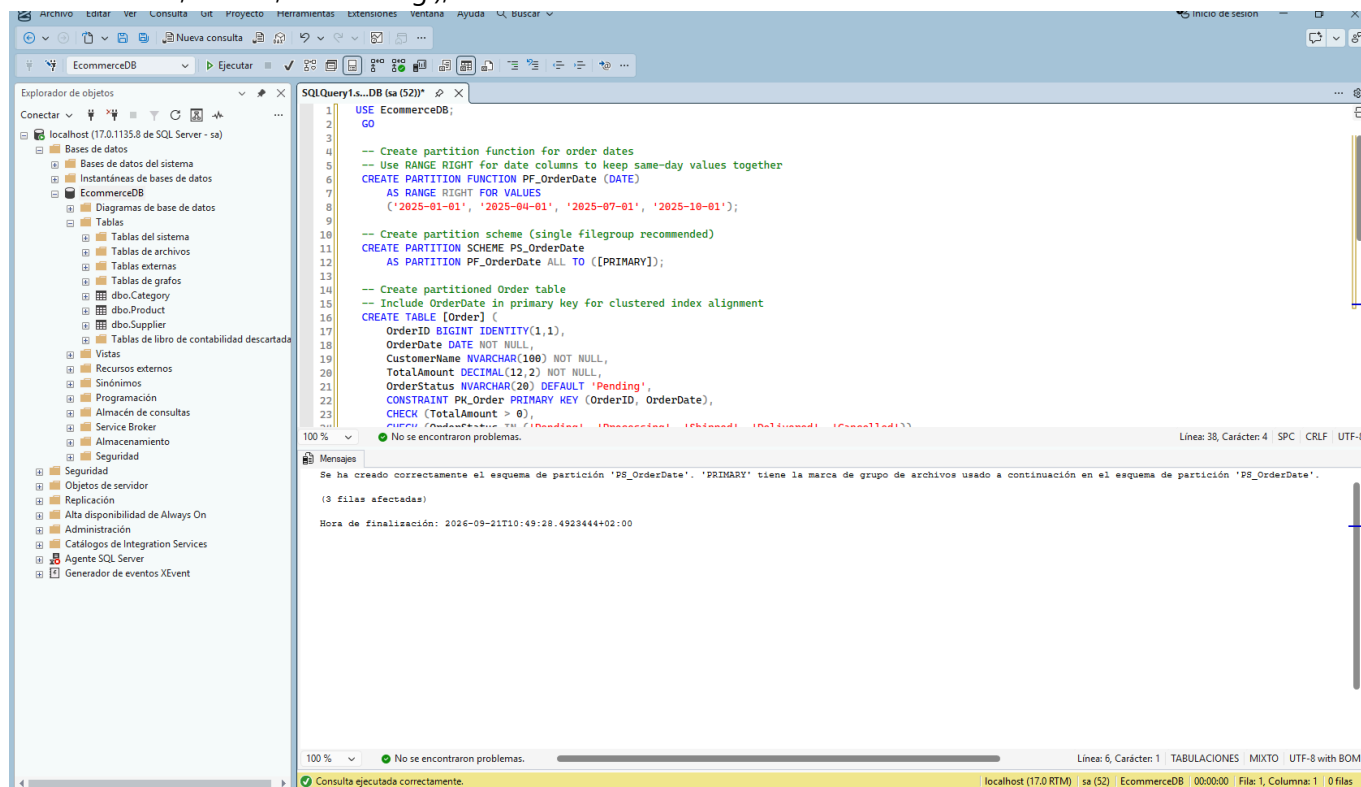
-- Create partition scheme (single filegroup recommended)
 CREATE PARTITION SCHEME PS_OrderDate AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);

-- Create partitioned Order table -- Include OrderDate in primary key for clustered index alignment
 CREATE TABLE [Order] (OrderID BIGINT IDENTITY(1,1), OrderDate DATE NOT NULL, CustomerName NVARCHAR(100) NOT NULL, TotalAmount DECIMAL(12,2) NOT NULL, OrderStatus NVARCHAR(20) DEFAULT 'Pending',
 CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate), CHECK (TotalAmount > 0), CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))) ON PS_OrderDate(OrderDate);

-- Create partitioned index
 CREATE NONCLUSTERED INDEX IX_Order_Customer ON Order ON PS_OrderDate(OrderDate); GO

-- Insert sample orders
 INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES ('2025-01-15', 'John Smith', 299.97, 'Delivered'), ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'), ('2025-06-10',

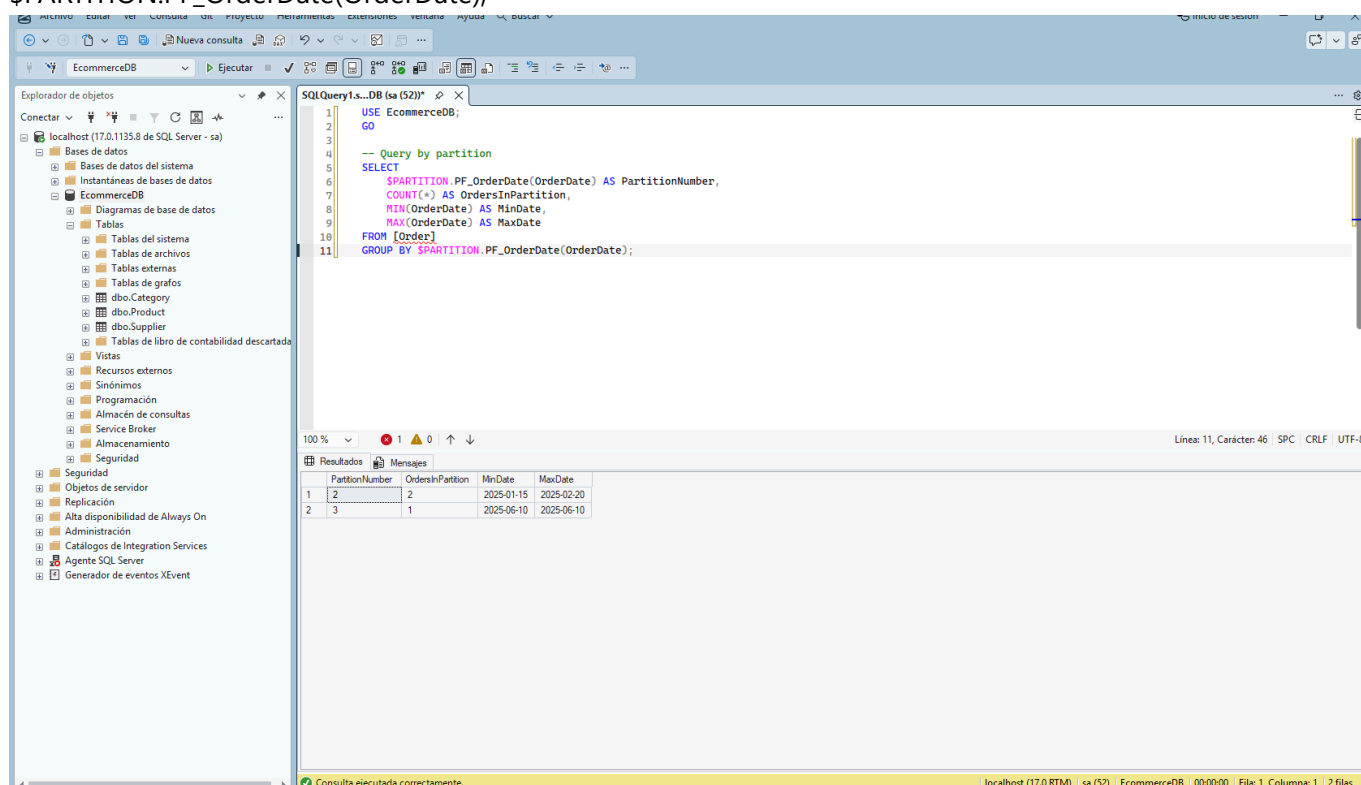
'Bob Johnson', 449.95, 'Processing'); GO



Ahora las consultamos

USE EcommerceDB; GO

-- Query by partition SELECT \$PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber, COUNT(*) AS OrdersInPartition, MIN(OrderDate) AS MinDate, MAX(OrderDate) AS MaxDate FROM [Order] GROUP BY \$PARTITION.PF_OrderDate(OrderDate);



7º paso

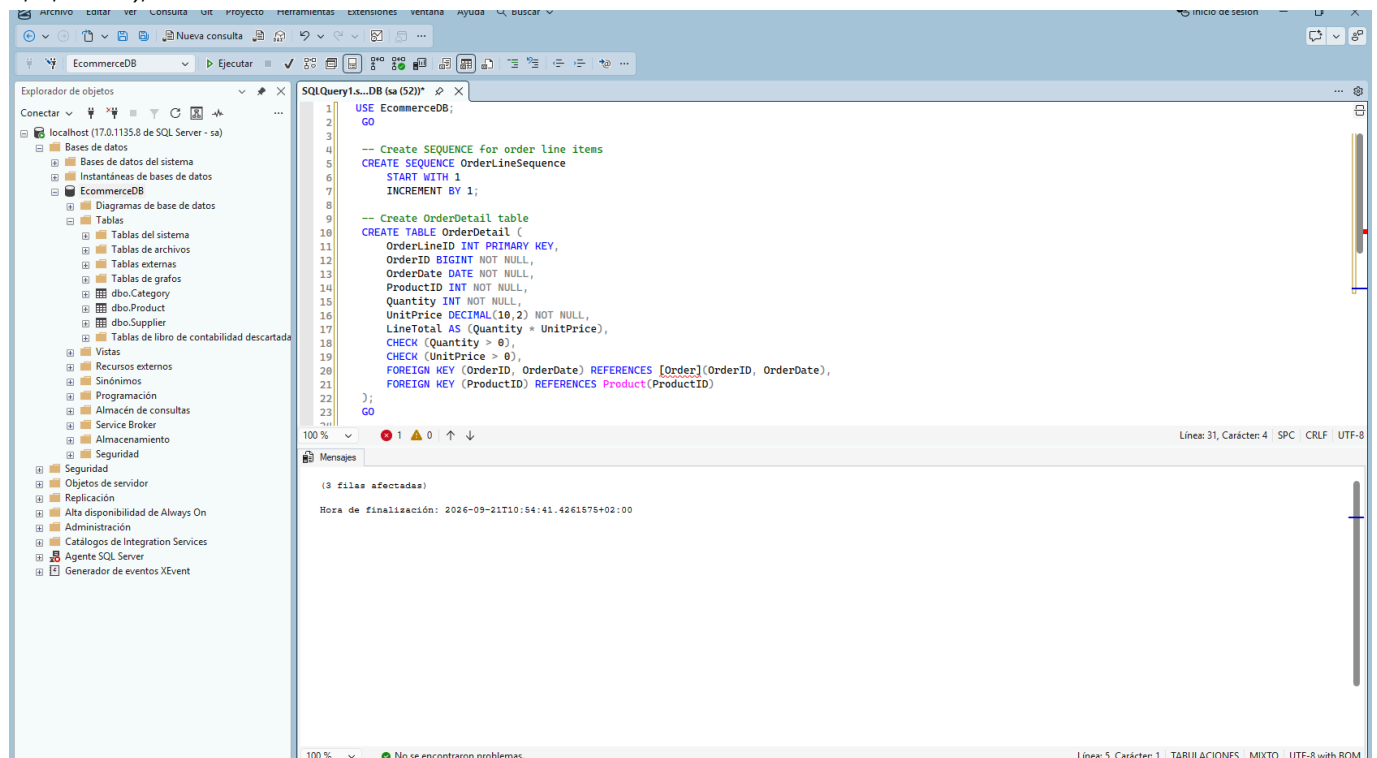
Creamos detalles del pedido con SEQUENCE. Las secuencias generan números únicos independientemente de cualquier tabla. Esta tarea utiliza una secuencia para los identificadores de los artículos de la línea de pedido.

USE EcommerceDB; GO

-- Create SEQUENCE for order line items CREATE SEQUENCE OrderLineSequence START WITH 1 INCREMENT BY 1;

-- Create OrderDetail table CREATE TABLE OrderDetail (OrderLineID INT PRIMARY KEY, OrderID BIGINT NOT NULL, OrderDate DATE NOT NULL, ProductID INT NOT NULL, Quantity INT NOT NULL, UnitPrice DECIMAL(10,2) NOT NULL, LineTotal AS (Quantity * UnitPrice), CHECK (Quantity > 0), CHECK (UnitPrice > 0), FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate), FOREIGN KEY (ProductID) REFERENCES Product(ProductID)); GO

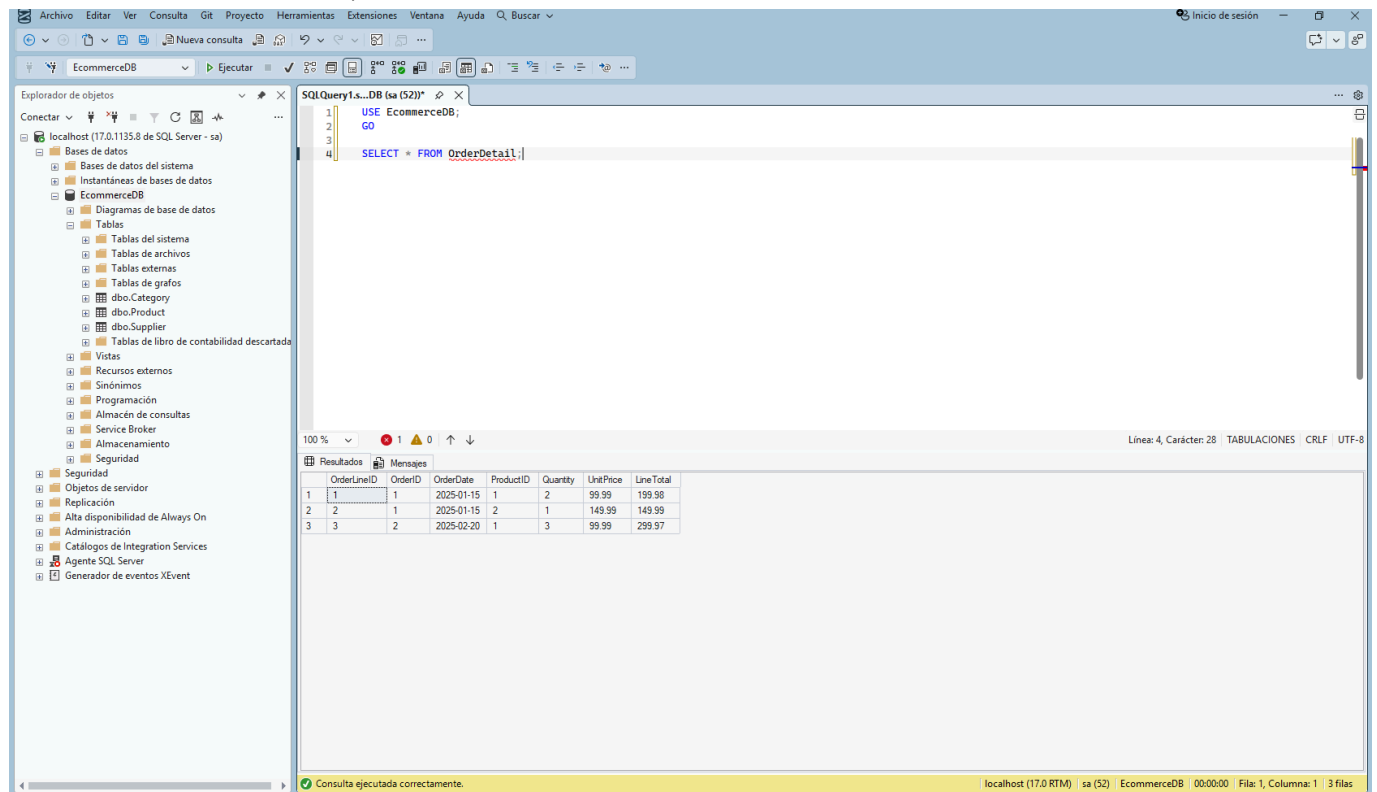
-- Insert order details using SEQUENCE INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice) VALUES (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99), (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99), (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99); GO



Ahora consultamos las secuencias

USE EcommerceDB; GO

```
SELECT * FROM OrderDetail;
```



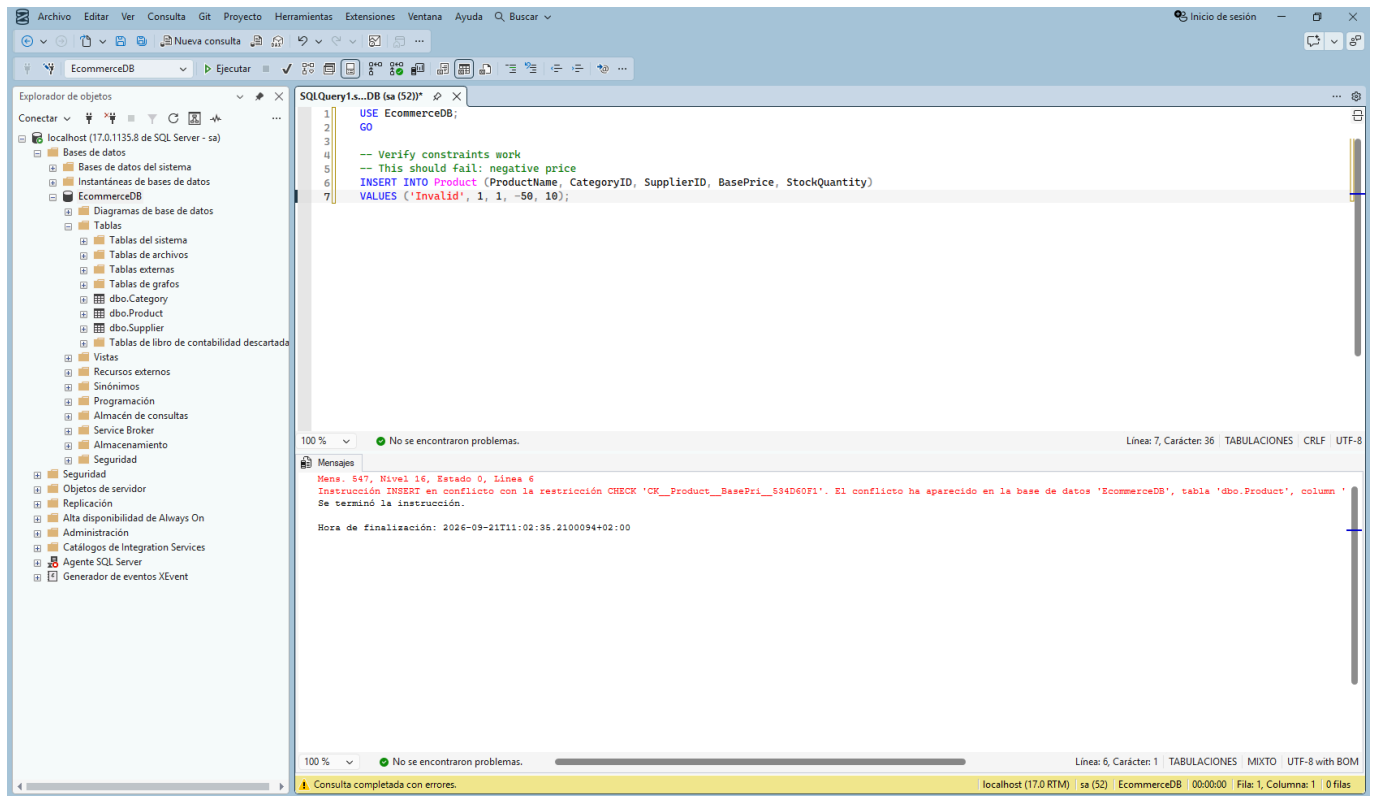
8º paso

En este último paso verificaremos los objetos de la base de datos, esto se hace para asegurarse de que todos los objetos de la base de datos se hayan creado correctamente.

Esta consulta debería fallar con una violación de restricción CHECK, lo que confirma que la restricción está funcionando correctamente.

USE EcommerceDB; GO

-- Verify constraints work -- This should fail: negative price INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity) VALUES ('Invalid', 1, 1, -50, 10);



Ahora verificamos el JSON y las consultas de particionamiento.

USE EcommerceDB; GO

-- Verify JSON queries work SELECT ProductName, JSON_VALUE(Metadata, '\$.color') AS Color FROM Product WHERE Metadata IS NOT NULL;

-- Verify partitioning SELECT \$PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount FROM [Order] GROUP BY \$PARTITION.PF_OrderDate(OrderDate);

-- Verify temporal table SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime FROM ProductPrice FOR SYSTEM_TIME ALL ORDER BY ProductID, SysStartTime;

